Artificial Intelligence

Assignment 6

Group: Ritchie

1. Eshwar Reddy Poreddy: 221100642
(eshwaraa07@uni-koblenz.de)

2. Prajna Shetty: 222100479
(prajnashetty73@uni-koblenz.de)

3. Vamshidhar Pratap: 222202552
(vamshidharpratap@uni-koblenz.de)

4. Yangsun Kim: 220202400
(ykim@uni-koblenz.de)

1 Answer Set Programming (10 Points)

Let P be the following extended logic program:

$$
\begin{aligned}
P: \quad & a \leftarrow not\ c. \\
& c \leftarrow b, not\ q. \\
& c \leftarrow not\ d, not\ e. \\
& e \leftarrow a, not\ d. \\
& d \leftarrow q. \\
& b.
\end{aligned}
$$

a) Compute the reduct P^S of state $S = \{a, b\}$. Justify why S is not an answer set for P.

b) Find all answer sets for P. Always mention the state S, its reduct P^S and justify why S is an answer set.

c) Transform the program into an equivalent default theory $T = (W, \Delta)$ such that there is a one to one correspondence between extensions and answer sets.

a. P^s = { a., c <- b., c., e <- a., d <- q. , b. }

Minimal model of P^s is : { a, b, c, d, e } which is ≠ S.
As S is not equal , it is not an answer set for P

b. S1 = { b, c, d }

Reduct of s1, P^{s1} = { c <-b. , d <- q., b. }
Minimal model of P^{s1} is {c,d,b} = S1

c. W = { a,b,c,d,e,q}

Δ = { δ1 = $\frac{:\sim c}{a}$,

δ2 = $\frac{T:\sim c}{a}$,

δ3 = $\frac{b:\sim q}{c}$,

δ4 = $\frac{T:\sim d,\sim e,}{c}$,

δ5 = $\frac{a:\sim d}{e}$,

δ6 = $\frac{q:T}{d}$,

δ7 = $\frac{T:T}{b}$, }

2 Default Resoning in Potassco (10 Points)

1. Write a constraint stating a literal and its contrary cannot both hold at the same time.

   :- holds(L, T), holds(neg(L), T).

2. Write a rule stating the closed world hypothesis: if a literal holds at a point in time and we cannot prove that its contrary literal holds at the next point in time, then the literal continues to hold.

   holds(L, T+1) :- holds(L, T), not holds(neg(L), T+1), time(T), time(T+1).

3. Write rules for a predicate possible/2 that describes whether it is possible to perform each action at a point in time. This corresponds to the C set in STRIPS.

   possible(load, T) :- holds(walking, T), not holds(loaded, T).
   possible(shoot, T) :- holds(loaded, T), holds(walking, T), not holds(dead, T).

   -performed(A, T) :- action(A), time(T), not possible(A, T).

4. Of course, we do not perform an action if we cannot prove that it is possible to perform it:

   -performed(A,T) :- action(A), time(T), not possible(A,T).

5. Write rules describing the effects of performing each action.

   holds(loaded, T+1) :- performed(load, T), time(T), time(T+1).
   holds(dead, T+1) :- performed(shoot, T), time(T), time(T+1).

6. Execute the program. How can you see from the output what will happen?

clingo version 5.7.0

Reading from stdin

-:15:23-41: info: atom does not occur in any rule head:

  performed(load,T)

-:16:21-40: info: atom does not occur in any rule head:

  performed(shoot,T)

Solving...

Answer: 1

holds(walking,0) holds(neg(loaded),0) holds(neg(dead),0) possible(load,0) holds(neg(dead),1)
holds(neg(loaded),1) holds(walking,1) holds(walking,2) holds(neg(loaded),2) holds(neg(dead),2)
holds(neg(dead),3) holds(neg(loaded),3) holds(walking,3) holds(walking,4) holds(neg(loaded),4)
holds(neg(dead),4) holds(neg(dead),5) holds(neg(loaded),5) holds(walking,5) holds(walking,6)
holds(neg(loaded),6) holds(neg(dead),6) holds(neg(dead),7) holds(neg(loaded),7)
holds(walking,7) holds(walking,8) holds(neg(loaded),8) holds(neg(dead),8) holds(neg(dead),9)
holds(neg(loaded),9) holds(walking,9) holds(walking,10) holds(neg(loaded),10)
holds(neg(dead),10) holds(neg(dead),11) holds(neg(loaded),11) holds(walking,11)
holds(walking,12) holds(neg(loaded),12) holds(neg(dead),12) holds(neg(dead),13)
holds(neg(loaded),13) holds(walking,13) holds(walking,14) holds(neg(loaded),14)
holds(neg(dead),14) holds(neg(dead),15) holds(neg(loaded),15) holds(walking,15)
holds(walking,16) holds(neg(loaded),16) holds(neg(dead),16) holds(neg(dead),17)
holds(neg(loaded),17) holds(walking,17) holds(walking,18) holds(neg(loaded),18)
holds(neg(dead),18) holds(neg(dead),19) holds(neg(loaded),19) holds(walking,19)
holds(walking,20) holds(neg(loaded),20) holds(neg(dead),20) holds(neg(dead),21)
holds(neg(loaded),21) holds(walking,21) holds(walking,22) holds(neg(loaded),22)
holds(neg(dead),22) holds(neg(dead),23) holds(neg(loaded),23) holds(walking,23)
holds(walking,24) holds(neg(loaded),24) holds(neg(dead),24) holds(neg(dead),25)
holds(neg(loaded),25) holds(walking,25) holds(walking,26) holds(neg(loaded),26)
holds(neg(dead),26) holds(neg(dead),27) holds(neg(loaded),27) holds(walking,27)
holds(walking,28) holds(neg(loaded),28) holds(neg(dead),28) holds(neg(dead),29)
holds(neg(loaded),29) holds(walking,29) holds(walking,30) holds(neg(loaded),30)
holds(neg(dead),30) holds(neg(dead),31) holds(neg(loaded),31) holds(walking,31)
holds(walking,32) holds(neg(loaded),32) holds(neg(dead),32) holds(neg(dead),33)
holds(neg(loaded),33) holds(walking,33) holds(walking,34) holds(neg(loaded),34)
holds(neg(dead),34) holds(neg(dead),35) holds(neg(loaded),35) holds(walking,35)

holds(walking,36) holds(neg(loaded),36) holds(neg(dead),36) holds(neg(dead),37)
holds(neg(loaded),37) holds(walking,37) holds(walking,38) holds(neg(loaded),38)
holds(neg(dead),38) holds(neg(dead),39) holds(neg(loaded),39) holds(walking,39)
holds(walking,40) holds(neg(loaded),40) holds(neg(dead),40) holds(neg(dead),41)
holds(neg(loaded),41) holds(walking,41) holds(walking,42) holds(neg(loaded),42)
holds(neg(dead),42) holds(neg(dead),43) holds(neg(loaded),43) holds(walking,43)
holds(walking,44) holds(neg(loaded),44) holds(neg(dead),44) holds(neg(dead),45)
holds(neg(loaded),45) holds(walking,45) holds(walking,46) holds(neg(loaded),46)
holds(neg(dead),46) holds(neg(dead),47) holds(neg(loaded),47) holds(walking,47)
holds(walking,48) holds(neg(loaded),48) holds(neg(dead),48) holds(neg(dead),49)
holds(neg(loaded),49) holds(walking,49) holds(walking,50) holds(neg(loaded),50)
holds(neg(dead),50) holds(neg(dead),51) holds(neg(loaded),51) holds(walking,51)
holds(walking,52) holds(neg(loaded),52) holds(neg(dead),52) holds(neg(dead),53)
holds(neg(loaded),53) holds(walking,53) holds(walking,54) holds(neg(loaded),54)
holds(neg(dead),54) holds(neg(dead),55) holds(neg(loaded),55) holds(walking,55)
holds(walking,56) holds(neg(loaded),56) holds(neg(dead),56) holds(neg(dead),57)
holds(neg(loaded),57) holds(walking,57) holds(walking,58) holds(neg(loaded),58)
holds(neg(dead),58) holds(neg(dead),59) holds(neg(loaded),59) holds(walking,59)
holds(walking,60) holds(neg(loaded),60) holds(neg(dead),60) holds(neg(dead),61)
holds(neg(loaded),61) holds(walking,61) holds(walking,62) holds(neg(loaded),62)
holds(neg(dead),62) holds(neg(dead),63) holds(neg(loaded),63) holds(walking,63)
holds(walking,64) holds(neg(loaded),64) holds(neg(dead),64) holds(neg(dead),65)
holds(neg(loaded),65) holds(walking,65) holds(walking,66) holds(neg(loaded),66)
holds(neg(dead),66) holds(neg(dead),67) holds(neg(loaded),67) holds(walking,67)
holds(walking,68) holds(neg(loaded),68) holds(neg(dead),68) holds(neg(dead),69)
holds(neg(loaded),69) holds(walking,69) holds(walking,70) holds(neg(loaded),70)
holds(neg(dead),70) holds(neg(dead),71) holds(neg(loaded),71) holds(walking,71)
holds(walking,72) holds(neg(loaded),72) holds(neg(dead),72) holds(neg(dead),73)
holds(neg(loaded),73) holds(walking,73) holds(walking,74) holds(neg(loaded),74)
holds(neg(dead),74) holds(neg(dead),75) holds(neg(loaded),75) holds(walking,75)
holds(walking,76) holds(neg(loaded),76) holds(neg(dead),76) holds(neg(dead),77)
holds(neg(loaded),77) holds(walking,77) holds(walking,78) holds(neg(loaded),78)
holds(neg(dead),78) holds(neg(dead),79) holds(neg(loaded),79) holds(walking,79)
holds(walking,80) holds(neg(loaded),80) holds(neg(dead),80) holds(neg(dead),81)
holds(neg(loaded),81) holds(walking,81) holds(walking,82) holds(neg(loaded),82)
holds(neg(dead),82) holds(neg(dead),83) holds(neg(loaded),83) holds(walking,83)
holds(walking,84) holds(neg(loaded),84) holds(neg(dead),84) holds(neg(dead),85)
holds(neg(loaded),85) holds(walking,85) holds(walking,86) holds(neg(loaded),86)
holds(neg(dead),86) holds(neg(dead),87) holds(neg(loaded),87) holds(walking,87)
holds(walking,88) holds(neg(loaded),88) holds(neg(dead),88) holds(neg(dead),89)
holds(neg(loaded),89) holds(walking,89) holds(walking,90) holds(neg(loaded),90)
holds(neg(dead),90) holds(neg(dead),91) holds(neg(loaded),91) holds(walking,91)
holds(walking,92) holds(neg(loaded),92) holds(neg(dead),92) holds(neg(dead),93)
holds(neg(loaded),93) holds(walking,93) holds(walking,94) holds(neg(loaded),94)

holds(neg(dead),94) holds(neg(dead),95) holds(neg(loaded),95) holds(walking,95)
holds(walking,96) holds(neg(loaded),96) holds(neg(dead),96) holds(neg(dead),97)
holds(neg(loaded),97) holds(walking,97) holds(walking,98) holds(neg(loaded),98)
holds(neg(dead),98) holds(neg(dead),99) holds(neg(loaded),99) holds(walking,99)
possible(load,1) possible(load,2) possible(load,3) possible(load,4) possible(load,5)
possible(load,6) possible(load,7) possible(load,8) possible(load,9) possible(load,10)
possible(load,11) possible(load,12) possible(load,13) possible(load,14) possible(load,15)
possible(load,16) possible(load,17) possible(load,18) possible(load,19) possible(load,20)
possible(load,21) possible(load,22) possible(load,23) possible(load,24) possible(load,25)
possible(load,26) possible(load,27) possible(load,28) possible(load,29) possible(load,30)
possible(load,31) possible(load,32) possible(load,33) possible(load,34) possible(load,35)
possible(load,36) possible(load,37) possible(load,38) possible(load,39) possible(load,40)
possible(load,41) possible(load,42) possible(load,43) possible(load,44) possible(load,45)
possible(load,46) possible(load,47) possible(load,48) possible(load,49) possible(load,50)
possible(load,51) possible(load,52) possible(load,53) possible(load,54) possible(load,55)
possible(load,56) possible(load,57) possible(load,58) possible(load,59) possible(load,60)
possible(load,61) possible(load,62) possible(load,63) possible(load,64) possible(load,65)
possible(load,66) possible(load,67) possible(load,68) possible(load,69) possible(load,70)
possible(load,71) possible(load,72) possible(load,73) possible(load,74) possible(load,75)
possible(load,76) possible(load,77) possible(load,78) possible(load,79) possible(load,80)
possible(load,81) possible(load,82) possible(load,83) possible(load,84) possible(load,85)
possible(load,86) possible(load,87) possible(load,88) possible(load,89) possible(load,90)
possible(load,91) possible(load,92) possible(load,93) possible(load,94) possible(load,95)
possible(load,96) possible(load,97) possible(load,98) possible(load,99)

SATISFIABLE

Models : 1

Calls : 1

Time : 0.052s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)

CPU Time : 0.000s